



DEPLOYMENT GUIDELINES

366Pi

Abstract

Deployment guideline for deployment of solutions to various environments

Abhijeet Dasgupta
abhijeet@366pitech.com

Date	Version	Author	Remarks
13/09/2023	0.1	Abhijeet Das Gupta	First Draft

Contents

Introduction	3
Key Objectives	3
Audience	3
Process Overview	4
1. Branching Strategy	4
2. Development Workflow	4
3. Feature Development	4
4. Code Review	4
5. Staging Deployment	4
6. Testing	4
7. Deployment Approval	5
8. Production Deployment	5
9. Production Testing	5
10. Merging to Main	5
11. Release Complete	5
Sample Email Templates	6
1. Deployment Approval Request Email	6
2. Approval mail from Approver	6
3. Production Testing Results mail	7
Changelog Template	8
Test Case Template	9
Test Report Template	10
Release and Deployment Plan Template	12

Introduction

This document outlines the comprehensive workflow that our team follows for managing software projects hosted on GitHub. Our aim is to ensure seamless collaboration, maintain code quality, and enable reliable and controlled deployments. This process not only enhances our development capabilities but also fosters a culture of accountability and transparency within the team.

Key Objectives

The primary objectives of our Software Development and Deployment Process are as follows:

1. **Efficient Collaboration:** By establishing a clear and structured process, we aim to facilitate collaboration among developers, testers, and other team members. This promotes a smoother and more coordinated workflow.
2. **Code Quality:** We prioritize code quality and consistency to ensure that our software is maintainable and scalable. Code reviews and best practices are integral to achieving this goal.
3. **Reliable Deployments:** Our staging and production deployment procedures are designed to minimize the risk of disruptions to our live systems. Thorough testing and approval processes are in place to maintain system reliability.
4. **Transparency and Accountability:** Transparency is crucial for effective project management. Our process ensures that all stakeholders have visibility into the development and deployment phases. Accountability is established through clear roles and responsibilities.

Audience

This document is intended for all team members involved in the software development and deployment process. This includes developers, testers, and other relevant stakeholders. It serves as a reference guide for understanding the workflow, roles, and responsibilities within our software development projects.

Process Overview

1. Branching Strategy

- **Main Branch:** This branch serves as the primary branch and contains the production-ready code. It is the stable version of your software that is currently running in the production environment. Changes to this branch should only be made after thorough testing and approval.
- **Staging Branch:** The staging branch is where code is deployed for testing in a staging environment that closely resembles the production environment. It allows for final validation of new features and bug fixes before they are deployed to production.
- **Feature/Branch Per Task:** For every new feature or bug fix, developers create separate branches. These branches are isolated workspaces where they can develop and test the specific changes. Each branch should have a clear and descriptive name, such as "feature/add-user-authentication" or "bugfix/fix-login-issue."

2. Development Workflow

- **Frequent Commits and Pushes:** Developers are encouraged to make frequent commits to their local branches and regularly push these commits to their remote branches on GitHub. This practice helps prevent code loss and allows for better collaboration among team members.

3. Feature Development

- **Feature/Branch Per Task:** When working on a new feature or fixing a bug, developers create separate branches dedicated to that task. This isolation ensures that changes do not interfere with other ongoing development efforts.
- **Collaborative Development:** Multiple developers can collaborate on the same feature or bug fix by creating their separate branches. This enables parallel work on different aspects of a project.
- **Pull Requests:** To integrate code changes into the feature branch, developers create pull requests. Pull requests serve as a mechanism for code review and approval before changes are merged.

4. Code Review

- **Team Lead/Owner Approval:** Before code changes are merged, they undergo a code review process. Team leads or solution owners review the code, provide feedback, and approve the changes if they meet quality and coding standards.

5. Staging Deployment

- **Merging to Staging:** Once code changes are approved, they are merged into the staging branch. The staging branch serves as a pre-production environment for final testing.
- **Deployment to Staging Environment:** The code changes in the staging branch are deployed to a staging environment that closely mirrors the production environment.

6. Testing

- **Comprehensive Testing:** Testing teams perform comprehensive testing in the staging environment. This includes testing not only the new features but also the existing features to ensure that there are no regressions.

- **Test Reports and Bug Tracking:** Testers document their findings in test reports and log any identified issues or bugs in a bug tracking system, such as JIRA.

7. Deployment Approval

- **Request for Approval:** Once testing is complete and all issues are resolved, a formal email requesting production deployment approval is sent to the Solution Architect. This email includes detailed information about the changes, test results, release plan, and proposed deployment date and time.

8. Production Deployment

- **Deployment to Production:** If the deployment is approved, the development team proceeds to deploy the approved changes to the production environment.

9. Production Testing

- **Testing in Production:** The testing team validates the production environment to ensure that the deployed changes are functioning as expected and that there are no issues.
- **Test Results:** The results of production testing are documented, and any issues are communicated in the deployment approval email thread.

10. Merging to Main

- **Creating a Pull Request:** After successful production testing, a pull request is created to merge the changes from the staging branch into the main branch.
- **Branch Cleanup:** Once the merge is complete, child branches created for feature development or bug fixes are deleted to maintain a clean and manageable repository.

11. Release Complete

- **Completion of Deployment Loop:** The deployment loop is considered complete when the changes are successfully deployed to the production environment, tested, and merged into the main branch. The software is now updated with the latest features and bug fixes.

This detailed breakdown should provide a clear understanding of each step in the software development and deployment process. It ensures that every aspect of the workflow is well-defined and understood by all team members involved.

Sample Email Templates

1. Deployment Approval Request Email

Subject: Request for Production Deployment Approval

Dear [Approver_Name],

We have completed development and testing for the [Feature/Bug Name] and would like to request your approval for production deployment.

Details:

- Feature/Bug: [Feature/Bug Name]
- Changelogs: [Changelogs]
- Test Cases: [Test Cases]
- Test Report: [Test Report]
- Release and Deployment Plan: [Plan]
- Proposed Release Date/Time: [Date/Time]
- Downtime Required (if any): [Downtime]

Please review the above details and provide your approval at your earliest convenience.

Best regards,

[Your Name]

[Your Title]

2. Approval mail from Approver

Subject: Re: Request for Production Deployment Approval - Approved

Dear [Name],

I have reviewed the details for the [Feature/Bug Name], and I approve the production deployment.

Please proceed with the deployment process, and ensure thorough testing in the production environment.

Best regards,

[Approver Name]

3. Production Testing Results mail

Subject: Production Testing Results for [Feature/Bug Name]

Dear [Your Name],

The production testing for the [Feature/Bug Name] has been successfully completed, and I'm pleased to inform you that all tests have passed without any issues.

Test Results:

[Test Results Summary]

The production environment is now ready for use, and the changes have been merged into the main branch.

If you have any questions or need further information, please feel free to reach out.

Best regards,

[Testing Team Lead's Name]

[Testing Team Lead's Title]

Changelog Template

Changelog for [Project Name] - [Version Number]

Release Date: [Release Date]

New Features

- [Feature 1 Description]
- [Feature 2 Description]

Bug Fixes

- [Bug 1 Description]
- [Bug 2 Description]

Enhancements

- [Enhancement 1 Description]
- [Enhancement 2 Description]

Other Changes

- [Change 1 Description]
- [Change 2 Description]

Dependencies

- [Dependency 1]
- [Dependency 2]

Notes

[Any additional notes or important information]

Test Case Template

Test Cases for [Feature/Bug Name]

Project: [Project Name]

Version: [Version Number]

Date: [Date]

Test Case 1: [Test Case Name]

Test Description:

[Description of the test case]

Steps to Reproduce:

1. [Step 1]
2. [Step 2]
3. [Step 3]

Expected Result:

[What should happen]

Actual Result:

[What happened during testing]

Status:

[Pass/Fail]

Test Case 2: [Test Case Name]

[Repeat the structure for additional test cases]

Test Report Template

Test Report for [Feature/Bug Name]

Project: [Project Name]

Version: [Version Number]

Date: [Date]

Summary

- Total Test Cases: [Total Test Cases]
- Passed: [Passed Test Cases]
- Failed: [Failed Test Cases]
- In Progress: [Test Cases in Progress]
- Blocked: [Blocked Test Cases]

Test Results

Test Case 1: [Test Case Name]

- Test Description: [Description of the test case]
- Expected Result: [What should happen]
- Actual Result: [What happened during testing]
- Status: [Pass/Fail]

Test Case 2: [Test Case Name]

[Repeat the structure for additional test cases]

Defects Found

- [Defect 1 Description]
- [Defect 2 Description]

Test Environment

- Operating System: [OS Version]
- Browser: [Browser Name and Version]
- Database: [Database Name and Version]
- Additional Details: [Any other relevant details]

Test Execution Details

- Testers: [Names of Testers]
- Test Date: [Test Date]

- Test Duration: [Duration in Hours]

Conclusion

[Summary of the testing process and any additional remarks]

Release and Deployment Plan Template

Release and Deployment Plan for [Project Name] - [Version Number]

Release Date

[Proposed Release Date]

Downtime

[Expected Downtime (if any)]

Deployment Steps

1. Preparation

- Backup existing data and configurations.
- Notify stakeholders about the upcoming release.

2. Deployment to Staging Environment

- Deploy to staging servers for final testing.

3. Testing in Staging

- Conduct comprehensive testing in the staging environment.
- Resolve any issues or bugs.

4. Approval for Production Deployment

- Request approval from [Solution Architect/Reviewer Name].
- Share test cases, test reports, and changelogs.

5. Deployment to Production

- Deploy the new version to production servers.

6. Post-Deployment Testing

- Test the production environment thoroughly.
- Ensure all systems are operational.

7. Monitoring

- Monitor the production environment for any issues.
- Address and resolve any issues as they arise.

Rollback Plan

[If necessary, provide a plan for rolling back to the previous version in case of unforeseen issues.]

Contact Information

- Support Team Contact: [Support Contact Details]
- Deployment Team Contact: [Deployment Team Contact Details]